

UNIFECAF

Trabalho Unifecaf

Controle de qualidade de peças em linha de montagem

Parte teórica — Análise e discussão

O sistema se chama **Trabalho Unifecaf**. É um protótipo web, em Python e Flask, para o controle de qualidade de peças de uma linha de montagem. A interface é uma única página (`templates/index.html`), com a fonte Inter, fundo quase preto e acento laranja. Os cliques ficam em `static/js/app.js`; não há `onclick` no HTML, o que combina com a política de conteúdo que só aceita script da própria origem.

Problema e objetivo

Na inspeção manual, peso, cor e comprimento dependem de quem lê a medida e de quem anota o resultado. Um valor fora da faixa pode seguir para a expedição, e o motivo se perde quando a conferência fica em papel.

O objetivo é tirar a regra da memória do operador. Cada peça entra com identificador, peso, cor e comprimento. O programa decide na hora se ela é aprovada ou reprovada, guarda todos os motivos quando há falha e coloca a peça aprovada em uma caixa de 10. Quando a caixa enche, ela fecha e a próxima aprovada abre outra. O relatório reúne totais, motivos e caixas, em vez de folhas soltas.

Regras de inspeção e de armazenamento

A peça só é aprovada se as três condições valem ao mesmo tempo. Os limites estão em `app/qualidade.py`:

- peso de 95 g a 105 g, inclusive;
- cor azul ou verde, comparada sem diferenciar maiúsculas, depois de remover espaços nas pontas;
- comprimento de 10 cm a 20 cm, inclusive.

Cada falha gera um motivo, com o critério (`peso`, `cor` ou `comprimento`) e uma descrição. A lista vai inteira para a coluna `motivos`. A peça não para no primeiro erro: peso, cor e comprimento fora da faixa ficam os três registrados. A lista vazia significa aprovação.

Antes da inspeção, `app/servicos.py` recusa a entrada incompleta. O identificador é obrigatório, vai para maiúsculas, tem no máximo 40 caracteres e não pode conter espaço. Peso e comprimento precisam ser números maiores que zero e são arredondados para duas casas. A cor é obrigatória. Identificador repetido é recusado. No formulário a cor pode ser azul, verde, vermelha, amarela, preta ou branca; só azul e verde passam.

Peça reprovada no cadastro geral é gravada sem caixa. Peça aprovada entra na caixa de status `aberta`. Se não houver caixa aberta, o sistema abre a de número seguinte ($\text{MAX}(\text{numero}) + 1$), com capacidade 10. A consulta da caixa aberta trava a linha (`FOR UPDATE`) e fica com no

máximo uma caixa em enchimento: a de maior número. A ocupação é a quantidade de peças com aquele `caixa_id`. Ao chegar em 10, a caixa passa a `fechada` e recebe `fechada_em`. A próxima aprovada abre outra caixa.

A remoção apaga só a peça escolhida. A caixa não é excluída; caixa vazia permanece no histórico. Se a peça estava numa caixa fechada e a ocupação fica abaixo de 10, essa caixa reabre somente quando não existe outra aberta. Se já houver caixa em enchimento, a fechada continua fechada com a nova quantidade, para a linha não ter duas caixas abertas. Se a peça saiu de uma caixa que já estava aberta, a caixa segue aberta com a ocupação atualizada.

O modal **Gerenciar**, aberto em cada caixa, lista as peças daquela caixa. Dá para incluir uma peça direto nela (POST `/api/caixas/<id>/pecas`) ou tirar uma (DELETE `/api/caixas/<id>/pecas/<id>`). Nesse caminho a peça só é gravada se for aprovada e se ainda houver vaga. Peça reprovada não entra e também não é salva por esse formulário. Caixa cheia esconde o formulário e pede que se retire uma peça. Se a inclusão completar 10, aquela caixa fecha.

O relatório percorre as reprovadas e soma cada critério. A mesma peça conta em mais de um motivo quando falhou em mais de uma regra. Outra passagem monta as caixas e a ocupação. Caixas utilizadas são as fechadas mais uma, se a caixa aberta já tiver ao menos uma peça. A taxa de aprovação é aprovadas sobre o total, com uma casa decimal, ou zero quando não há peças.

Modelo de dados

O banco é MySQL ou MariaDB, nome `gestao_pecas`, conjunto de caracteres `utf8mb4` e collation `utf8mb4_unicode_ci`. O esquema está em `schema.sql` e é o mesmo criado por `app/db.py`. As tabelas usam InnoDB. A subida do programa executa `CREATE DATABASE IF NOT EXISTS` e `CREATE TABLE IF NOT EXISTS`. Não há `DROP`, `TRUNCATE` nem apagamento de linhas nessa inicialização.

caixas. `id` inteiro automático, `numero` inteiro único, `status` (`aberta` ou `fechada`, padrão `aberta`), `capacidade` (padrão 10), `criada_em` e `fechada_em` (nulo enquanto aberta).

usuarios. `id`, `nome` (até 80), `email` único (até 120), `senha_hash` (até 255), `ativo`, `refresh_token` (até 700, nulo sem sessão), `ultimo_login`, `falhas_login`, `bloqueado_ate` e `criado_em`. A senha em claro não é gravada.

eventos_seguranca. `id`, `tipo` (até 40), `email`, `ip` (até 64), `detalhe` (até 255) e `criado_em`. É o registro de auditoria do login, do bloqueio, da sessão suspeita, do logout e da troca de senha.

pecas. `id` texto até 40 (chave primária), `peso` e `comprimento` decimais (8,2), `cor` até 30, `status` (`aprovada` ou `reprovada`), `motivos` em JSON, `caixa_id` nulo com chave estrangeira para `caixas.id`, e `criada_em`.

Fluxo da aplicação e menu

O operador autentica e permanece na mesma página. O menu lateral repete as funções do desafio e acrescenta o painel:

1. **Painel** — totais, as três regras e as últimas inspeções. A barra lateral mostra a caixa em enchimento.
2. **Cadastrar peça** — modal com identificador, peso, cor e comprimento. A inspeção ocorre no servidor ao salvar.
3. **Peças** — tabela com abas Todas, Aprovadas e Reprovadas, busca por identificador ou cor, detalhe dos motivos e remoção da linha.
4. **Remover peça** — modal com a lista de peças cadastradas. A exclusão é `DELETE /api/pecas/<id>`.
5. **Caixas** — abas Fechadas (padrão) e Todas. Cada cartão tem o botão **Gerenciar**, que abre o modal da caixa.
6. **Relatório** — aprovadas, reprovadas, caixas fechadas, caixas usadas, motivos, peças reprovadas e ocupação. Há impressão.
7. **Como funciona** — leitura do código, na própria interface, com trechos reais de `app/qualidade.py`, `app/servicos.py`, `app/db.py`, `app/auth.py`, `app/seguranca.py` e `static/js/app.js`.

No topo ficam a busca, o botão **Exemplos** e **Nova peça**. Exemplos inclui um lote fixo de 16 peças (QL-001 a QL-012 aprovadas e QL-013 a QL-016 reprovadas). Identificador que já existe é mantido; nenhum dado atual é apagado. O menu da conta oferece **Trocar senha** e **Sair**.

As rotas de peças, caixas, relatório, resumo, demonstração, logout, dados do usuário e troca de senha exigem o JWT de acesso no cabeçalho `Authorization: Bearer`. Ficam públicas a página inicial, `POST /api/auth/login`, `POST /api/auth/refresh` e `GET /api/health`. O corpo da requisição tem limite de 256 KiB.

Segurança

O desenho separa um token de acesso curto de um token de renovação. Os dois são JWT HS256. A assinatura usa duas chaves distintas do Trabalho Unifecaf, gravadas no `.env` em `JWT_ACCESS_SECRET` e `JWT_REFRESH_SECRET`. Se uma delas estiver vazia, o programa gera um valor e grava no `.env`. Se as duas forem iguais, a chave de renovação é trocada por outra. Os valores não entram neste texto.

O token de acesso vale 15 minutos e vai no JSON da resposta, para o JavaScript enviá-lo como Bearer. O de renovação vale 7 dias e vai no cookie `refreshToken: HttpOnly, SameSite estrito`, caminho `/api/auth` e Secure quando `COOKIE_SECURE=true` ou, na falta dessa variável, quando `FLASK_ENV=production`. O JavaScript não lê esse cookie. Cada renovação emite outro par, grava o novo refresh no usuário e confere um resumo (SHA-256, 16 caracteres hexadecimais) do navegador e do IP. Se a origem mudar, o refresh é apagado e a sessão termina com código 403. Logout zera `refresh_token` e remove o cookie. Usuário com `ativo` igual a zero perde o acesso na hora, mesmo com token dentro do prazo.

A senha é conferida com bcrypt, custo 12. Só o hash fica no banco. Se o e-mail não existe, ou se o valor gravado não é um hash bcrypt, a comparação usa um hash fictício. Assim a resposta não fica mais rápida quando a conta não está cadastrada. E-mail inexistente e senha errada devolvem a mesma mensagem, "Credenciais inválidas."

Há dois limites independentes. Na memória do processo, o mesmo e-mail pode tentar o login no máximo 30 vezes em 15 minutos; acima disso a resposta é 429, antes da consulta da senha. No banco, cinco senhas erradas seguidas gravam `bloqueado_ate` para 15 minutos à frente e as tentativas seguintes, enquanto o prazo não passa, respondem que há tentativas demais. Login bem-sucedido zera `falhas_login` e o bloqueio.

Toda resposta recebe cabeçalhos em `aplicar_cabecalhos`: `Server` com o nome Trabalho Unifecaf, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Frame-Options: DENY` e `Permissions-Policy` sem câmera, microfone e geolocalização. A política de conteúdo restringe script, conexão, imagem, frame, base e formulário à própria origem; estilo pode ser local, inline ou da folha do Google Fonts, de onde também vem a Inter. `Strict-Transport-Security (max-age=31536000; includeSubDomains)` só sai do Flask quando `FLASK_ENV=production`. Caminho com `..` ou caractere nulo é recusado.

A auditoria insere em `eventos_seguranca` os tipos `login_ok`, `login_invalido`, `login_bloqueado`, `login_inativo`, `sessao_suspeita`, `logout` e `senha_alterada`, com e-mail, IP e um detalhe curto. Com `TRUST_PROXY=true`, o IP é o primeiro valor de `X-Forwarded-For`; sem isso, usa-se o endereço direto da conexão.

A troca de senha exige a senha atual e uma nova com no mínimo 8 caracteres, letra maiúscula, letra minúscula e número. Ao salvar, o hash é substituído, `refresh_token` fica nulo e o cookie é apagado: a sessão de renovação deixa de valer e é preciso entrar de novo. A senha inicial `trabalho123` não passa nessa regra, porque não tem maiúscula. Ela só existe porque a conta de avaliação foi semeada com esse valor. A política vale na troca, não na conferência do login dessa conta já gravada.

Se a tabela `usuarios` estiver vazia na primeira subida, o programa cria um administrador com `ADMIN_NOME`, `ADMIN_EMAIL` e `ADMIN_PASSWORD` do `.env`. Se já houver usuário, nada é inserido e a senha existente não é alterada.

Credenciais de avaliação

A conta de correção é:

- e-mail: `trabalho@unifecaf.com`
- senha: `trabalho123`

Os mesmos dados aparecem no card de login, em texto selecionável, e já vêm preenchidos nos campos. O nome gravado dessa conta no banco é o de `ADMIN_NOME`.

Publicação com Nginx

Há dois arquivos em `deploy/nginx/`. O `server_name` de exemplo é `trabalho-unifecaf.unifecaf.com`. O nome mostrado ao operador é Trabalho Unifecaf.

`trabalho-unifecaf-http.conf` escuta a porta 80. O caminho `/.well-known/acme-challenge/` serve o desafio do certificado. Qualquer outra URL responde 301 para HTTPS no mesmo host.

`trabalho-unifecaf-https.conf` escuta a porta 443 com TLS 1.2 e 1.3, encerra o certificado e encaminha para o Flask em `127.0.0.1:5000`. Envia `X-Forwarded-For` e `X-Forwarded-Proto`. O corpo máximo no proxy é 256k, alinhado ao limite do Flask. Esse bloco também envia HSTS, `nosniff`, `X-Frame-Options: DENY` e a política de referência.

Em produção o `.env` usa `FLASK_ENV=production`, `COOKIE_SECURE=true`, `TRUST_PROXY=true` e `BIND_HOST=127.0.0.1`, para o processo Python não ficar exposto direto na rede. O ajuste de domínio e dos caminhos do certificado fica no próprio arquivo do Nginx, conferido com `nginx -t` antes de recarregar.

Como executar localmente

É preciso Python 3.11 ou superior e MySQL ou MariaDB. No Windows, o XAMPP atende: o serviço MySQL precisa estar no ar. Nada neste roteiro apaga o banco.

Na pasta do projeto:

```
cd C:\Users\diego\trabalho5
python -m venv .env
.\.env\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
.\.env\Scripts\python.exe run.py
```

Se o `.env` já existir, ele deve ser mantido. Apagar esse arquivo ou esvaziar as chaves JWT faria o programa gerar segredos novos e invalidar sessões já emitidas. A aplicação sobe em <http://127.0.0.1:5000>. O banco `gestao_pecas` e as tabelas são criados só quando ainda não existem. Peças, caixas, usuários e eventos que já estão gravados permanecem.

O `.env.example` traz `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME` e `FLASK_PORT`. No XAMPP padrão, usuário `root` e senha vazia funcionam. Sem login, a API de operação responde 401.

Benefícios da solução

A inspeção deixa de depender da memória de quem confere. No cadastro, o Trabalho Unifecaf aplica as três regras na hora e devolve o resultado. Quando a peça é reprovada, ficam registrados todos os motivos, não apenas o primeiro. Corrigir o peso não esconde, depois, uma cor ou um comprimento fora da faixa.

A caixa de dez peças dispensa a contagem manual do lote. A aprovada entra na caixa aberta; na décima, a caixa fecha e a seguinte começa. A reprovada permanece no cadastro, sem ocupar vaga. O relatório reúne aprovadas, reprovadas por critério e caixas utilizadas. O identificador liga a peça à caixa e aos motivos, o que permite rastrear o que seguiu e o que ficou retido.

O protótipo também pode sair do laboratório. Em vez de uma página aberta, a operação exige autenticação e pode ser publicada atrás do Nginx, com o mesmo controle de sessão usado quando o sistema fica exposto na internet.

Desafios de desenvolvimento

A regra de negócio era direta. Peso, cor e comprimento são três condições independentes, a caixa fecha em um limite fixo e o relatório percorre as reprovações para somar cada critério. Essa parte apoiou-se em conhecimento já praticado. Não houve dificuldade relevante na faixa de 95 g a 105 g, na cor ou no comprimento.

O esforço real foi outro: deixar um protótipo acadêmico apto a ir para a internet, com o cuidado já usado em sistemas em produção. Esse repertório veio de experiência com VPS, segurança de aplicação e publicação, não de um primeiro contato com essas técnicas.

Os segredos que assinam os tokens ficam no `.env`, fora do código, e são distintos para acesso e renovação. O token curto permanece na memória do navegador. O de renovação, válido por sete dias, vai em cookie HttpOnly, com SameSite estrito e atributo Secure em produção. Cada renovação substitui o cookie anterior, de modo que o antigo deixa de valer. O login tem limite de tentativas por e-mail e bloqueia a conta após cinco falhas. A política de conteúdo aceita script somente da própria origem, sem código inline na página. Na frente do processo, o Nginx redireciona o HTTP para HTTPS, encerra o TLS 1.2 e 1.3 e encaminha o protocolo original. Com o proxy marcado como confiável, o aplicativo lê o IP encaminhado. O cookie Secure e o HSTS do Flask entram em produção; o bloco HTTPS do Nginx também envia HSTS. O detalhe desses controles está na seção de segurança e na de publicação.

Reflexão

O protótipo decide com dados digitados. Numa fábrica, peso e comprimento poderiam vir de sensor e a cor de uma câmera; a mesma função de avaliação continuaria no centro, mudando só a origem do número. Uma expansão ligaria o cadastro a um CLP ou a uma API da linha, para a peça aprovada seguir à caixa física e a reprovada à esteira de retrabalho. O fechamento aos 10 itens poderia acionar o fechamento da caixa. Visão computacional veria defeito de superfície que a regra atual não trata, e o histórico de reprovação poderia alimentar um painel da supervisão. Isso não está implementado. O que o sistema faz hoje é medir o que foi informado, decidir com critério explícito, guardar a reprovação com todos os motivos e armazenar a aprovada em lote de 10.